САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №2
по дисциплине «Методы оптимизации»
Тема: «Метод ломаных»

Выполнила:
Степанова Екатерина Андреевна, К3341
поток 1.2

Санкт-Петербург
2026 г.

Задание

Разработайте программу на языке Python или подобном (C++, Java...), которая будет решать задачу поиска глобального экстремума для заданной функции на заданном отрезке.

На вход программа принимает строку одномерной функции, например f(x) = x + sin(3.14159*x) (она заведомо должна являться липшицевой), координаты концов отрезка (вещественные числа), точность вычисления eps (например, 0,01). На выходе она выполняет визуализацию графика исходной функции, вспомогательных функций (по возможности итоговой ломаной, на которой достигнута требуемая точность), приближенное значение аргумента и минимального значения функции, число пробовавшихся итераций, потраченной время.

Программа должна быть либо в Jupiter Notebook или в репозитории. Должно быть продемонстрирована работа программы на тестовой функции (любой одномерный сред двумерной функции): функция Растригина, функция Экли и др., но с наличием не менее двух локальных минимумов на заданном отрезке.

Результаты расчетов должны отражаться в виде визуализации (функция, ломаная линия или вспомогательные функции, указание места найденного приближенного значения аргумента и соответствующей точки минимума), приближенные значения аргумента и функции, число итераций, затраченное время для расчетов.

Демонстрация должна быть отражена в PDF файле, который вместе ссылкой на файл с кодом, с файл визуализация и(или) файл Jupiter Notebook, размещается в Moodle в ответ на это задание.

Теория

Метод пиявского(ломаных) - метод нахождения глобального минимума(максимума) липшицевой функции.

Функция W(x) называется Липшицевой, если при любых u,v∈[a,b] найдется константа L>0 , при которой выполняется:

$$|W(u) - W(v)| \leq L|u - v|.$$

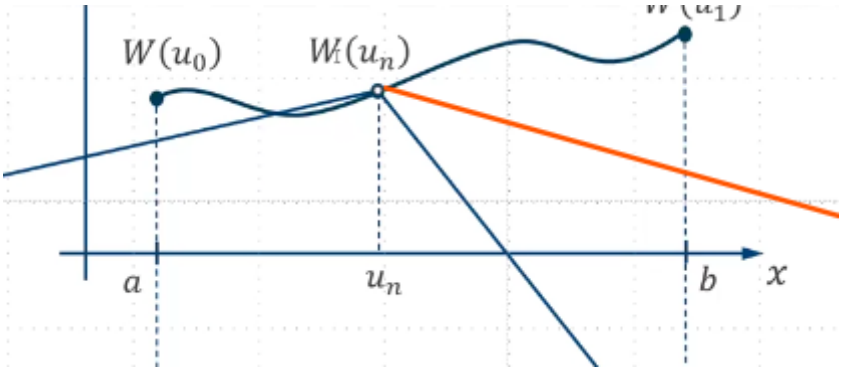
Иными словами при любых u,v∈[a,b] , прирост функции между этими точками по модулю будет МЕНЬШЕ, чем прирост самого аргумента по модулю, помноженного на константу.

Основной конструкционный элемент:

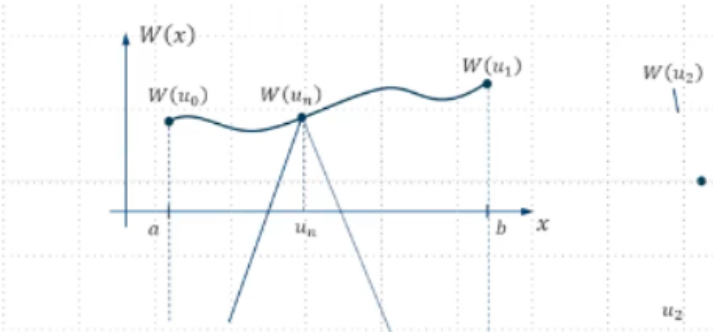
$$g_n(x, u_n) = W(u_n) - L|x - u_n|$$

g - звено ломаной - основной конструкционный элемент, из которого собирается итоговая сложная функция(является одной из многих деталей, которые, пересекаясь, образуют непрерывную прямую)

Число L называется *константой Липшица*.
Геометрически константа Липшица определяет наклон основного конструкционного элемента.
Если эту константу недооценить(слишком малая по модулю), то может быть такое, она пробьет нашу функцию, появятся искаженные решения(метод не будет работать)



Если L переоценить, то звено будет под более большим углом, метод будет работать, но дольше, а=появятся риски с вычислительной точностью.


W(x)
W(u0)
W(un)
W(u1)
W(u2)
a
un
b
x
u2

Выполнение

В отчете я предоставлю реализацию только важных, по моему мнению частей: нахождение константы Липшица и метод Пиявского(ломаных).

Метод Пиявского

```
def piyavsky_method(W, a, b, eps, L, max_iter=500):
    if not (np.isfinite(a) and np.isfinite(b)) or a >= b:
        raise ValueError("Требуется конечный отрезок с условием a < b")
    if not np.isfinite(eps) or eps <= 0:
        raise ValueError("eps должно быть конечным и положительным")
    if not np.isfinite(L) or L <= 0:
        raise ValueError("L должно быть конечным и положительным")
    if not isinstance(max_iter, (int, np.integer)) or max_iter <= 0:
        raise ValueError("max_iter должно быть положительным целым числом")

    time_start = time.perf_counter()
    u = [float(a), float(b)]
    Wu = [float(W(a)), float(W(b))]
    if not np.all(np.isfinite(Wu)):
        raise ValueError("Функция должна иметь конечные значения на концах отрезка")

    converged = False
    delta = np.inf
    lower = []

    for iteration in range(1, max_iter + 1):
        lower = []
        for i in range(len(u) - 1):
            u_cross = (Wu[i] - Wu[i + 1]) / (2 * L) + (u[i] + u[i + 1]) / 2
```

```
                p_cross = (Wu[i] + Wu[i + 1]) / 2 - L * (u[i
    + 1] - u[i]) / 2
                lower.append((p_cross, u_cross, i))

            p_new, u_new, i = min(lower)
            W_n = float(W(u_new))
            if not (np.isfinite(p_new) and
    np.isfinite(u_new) and np.isfinite(W_n)):
                raise ValueError("Получено нечисловое
    значение функции или ломаной")

            delta = W_n - p_new
            tol = 1e-12 * max(1.0, abs(W_n), abs(p_new))
            if delta < -tol:
                raise ValueError(
                    f"Недостаточная константа Липшица
    L={L:g}: "
                    f"получен отрицательный разрыв
    W(u_new)-p_new={delta:g}"
                )

            u.insert(i + 1, u_new)
            Wu.insert(i + 1, W_n)

            if delta < eps:
                converged = True
                break

        k = int(np.argmin(Wu))
        time_end = time.perf_counter()
        status = "converged" if converged else
    "max_iter_reached"

        return {
            "u_star": u[k],
            "W_star": Wu[k],
            "iterations": iteration,
            "delta": delta,
            "time": time_end - time_start,
            "u": u,
            "Wu": Wu,
```

```
            "lower": [(x, p) for p, x, _ in lower],
            "converged": converged,
            "status": status,
        }
```

На вход функция принимает:
W - данную функцию
a, b - рассматриваемый отрезок
eps - точность вычисления
L - константа Липшица
max_iter - количество максимальных итераций(для отслеживания)

Алгоритм:
На каждой итерации программа:

1. рассматривает все соседние верхние вершины;
2. вычисляет координаты точек пересечения;
3. вычисляет значения ломаной в этих точках;
4. выбирает минимальную нижнюю вершину;
5. вычисляет значение исходной функции в найденной точке;
6. проверяет условие окончания;
7. при необходимости добавляет новую точку;
8. повторяет процесс.

При этом поиск не заменяется перебором функции на мелкой сетке.

Это важно, поскольку в задании отдельно указано, что использование мелкого разбиения вместо расчёта точек пересечения и ведения верхних и нижних вершин является ошибочным подходом.

Представленный алгоритм реализован для случая нахождения глобального минимума, но если мы хотим найти глобальный максимум достаточно умножить функцию W(x) на коэффициент -1. таким образом, мы превратим задачу поиска максимума в задачу поиска минимума.

Поиск константы Липшица

По определению, константа Липшица должна удовлетворять неравенству для любых двух точек u, v на отрезке [a, b].

$$|W(u) - W(v)| <= L|u - v| \text{ -> } \frac{|W(u)-W(v)|}{|u-v|} <= L$$

т.е. верхняя граница наклона любой секущей(максимально возможная крутизна, с которой функция может расти на этом участке)

17.Теорема Лагранжа

Формула конечных приращений или теорема Лагра́нжа о среднем значении утверждает, что если функция непрерывна на отрезке и дифференцируема в интервале , то найдётся такая точка $c \in (a;b)$, что

$$\frac{f(b) - f(a)}{b - a} = f'(c).$$

Если функция дифференцируема, то по теореме Лагранжа о среднем значении, наклон секущей между любыми u и v равен производной в некоторой точке c между ними.

$$\frac{|W(u)-W(v)|}{|u-v|} = L = W'(c)$$

Так как точка c всегда лежит на нашем отрезке, наклон абсолютно любой секущей никогда не сможет превысить самое большое значение производной на этом отрезке.

Из чего следует, что минимально возможная константа L - максимум модуля ее производной.

$$L = max|W'(x)|$$

То есть L должна быть достаточно большой, чтобы ограничивать скорость изменения функции на всём исследуемом отрезке. В теории работы именно такая константа используется при построении вспомогательных функций.

```
1. def find_L(func_str, a, b, n=20001, reserve=1.2):
```

```
    """Численная оценка константы Липшица с коэффициентом
запаса."""
    if not (np.isfinite(a) and np.isfinite(b)) or a >= b:
        raise ValueError("Требуется конечный отрезок с условием a
< b")
    if not isinstance(n, (int, np.integer)) or n < 2:
        raise ValueError("n должно быть целым числом >= 2")
    if not np.isfinite(reserve) or reserve < 1:
        raise ValueError("reserve должно быть конечным и не
меньше 1")

    W = parse_function(func_str)
    xs = np.linspace(a, b, n)
    ys = np.asarray([W(x) for x in xs], dtype=float)
    if not np.all(np.isfinite(ys)):
        raise ValueError("Функция имеет нечисловые или
бесконечные значения на отрезке")

    slopes = np.abs(np.diff(ys) / np.diff(xs))
    raw_L = float(np.max(slopes))
    # Для постоянной функции raw_L == 0, но методу требуется L >
0
    return max(raw_L * reserve, np.finfo(float).eps)
```

Данная функция на заданном отрезке приближенно оценивает константу Липшица.

На вход функция принимает:
func_str - функция в виде строки, например "x + sin(pi*x)"
a, b - рассматриваемый отрезок
n - количество точек, в которых будет проверяться функция

В коде я по сути делаю то же самое, что описала выше, я решила посчитать все выражения, используя функцию

$$\frac{|W(u)-W(v)|}{|u-v|} = L = W'(c)$$

А затем взяла максимум.

Результаты

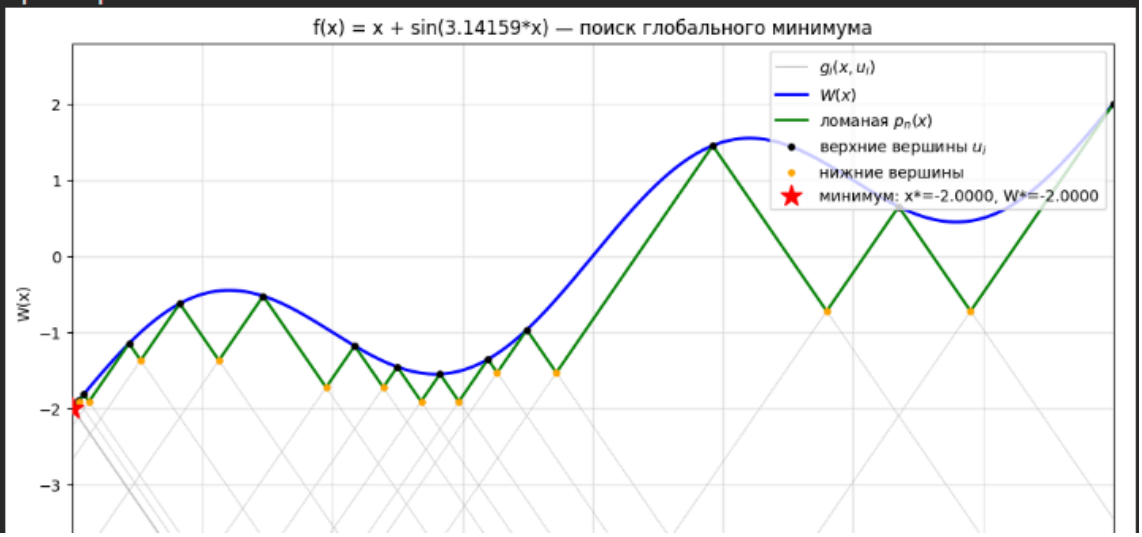
В качестве тестовых функция я взяла предложенную в методичке функцию, а также функцию Растригина и функция Экли.

Для глобального минимума.

Функция из методички

```
 x + sin(3.14159*x)
 x + sin(3.14159*x)
Автоматическая оценка L (reserve=1.2): 4.96990775195042

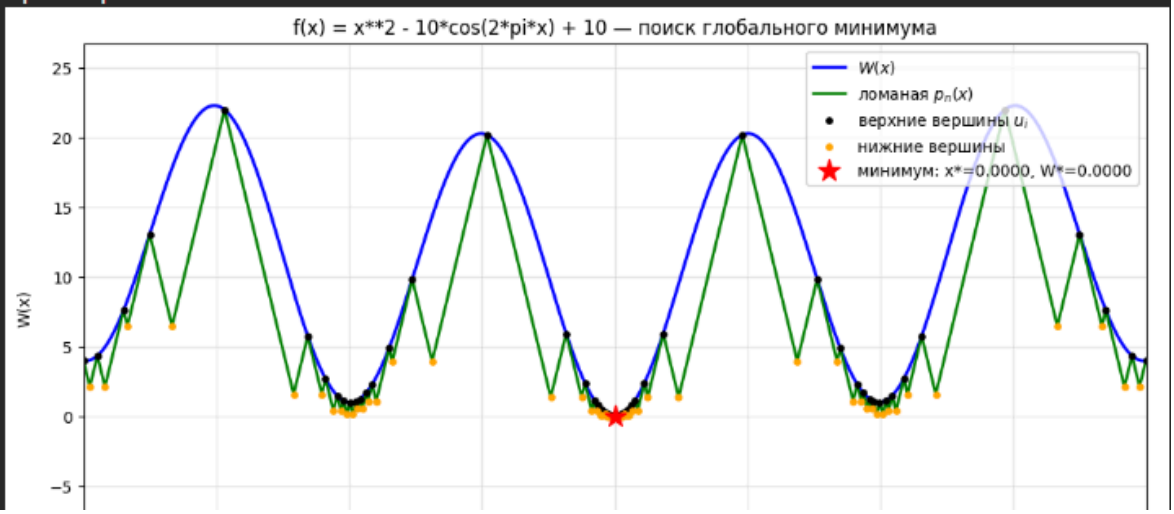
ГЛОБАЛЬНЫЙ МИНИМУМ
Функция: f(x) = x + sin(3.14159*x)
Отрезок: [-2, 2]
eps = 0.01
L = 4.96990775195042
x* = -2.000000
W(x*) = -1.999995
Число итераций = 14
Статус = converged
Разрыв W(u_new)-p_new = 0.00149266
Время расчёта = 0.54 мс
```



Функция Растригина

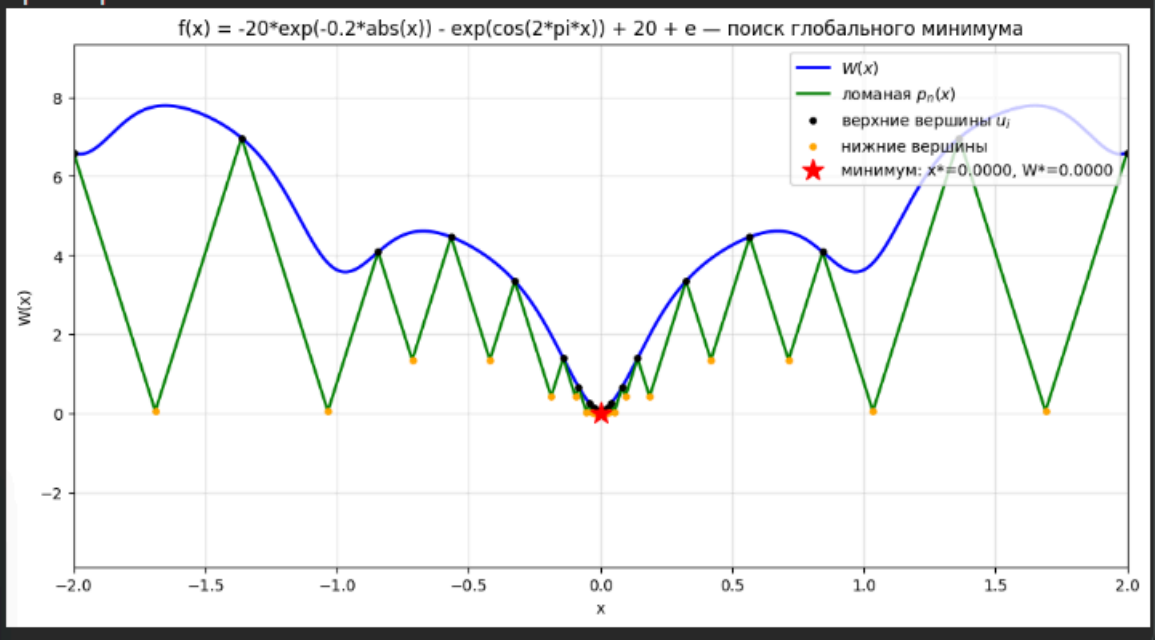
```
 x**2 - 10*cos(2*pi*x) + 10
 x**2 - 10*cos(2*pi*x) + 10
Автоматическая оценка L (reserve=1.2): 78.39917320442932

ГЛОБАЛЬНЫЙ МИНИМУМ
Функция: f(x) = x**2 - 10*cos(2*pi*x) + 10
Отрезок: [-2, 2]
eps = 0.01
L = 78.39917320442932
x* = 0.000000
W(x*) = 0.000000
Число итераций = 180
Статус = converged
Разрыв W(u_new)-p_new = 0.00977544
Время расчёта = 18.34 мс
```



Функция Экли

```
Функция: f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e
Отрезок: [-2, 2]
eps = 0.01
L = 21.1
x* = 0.000000
W(x*) = 0.000000
Число итераций = 32
Статус = converged
Разрыв W(u_new)-p_new = 0.00915031
Время расчёта = 1.83 мс
```



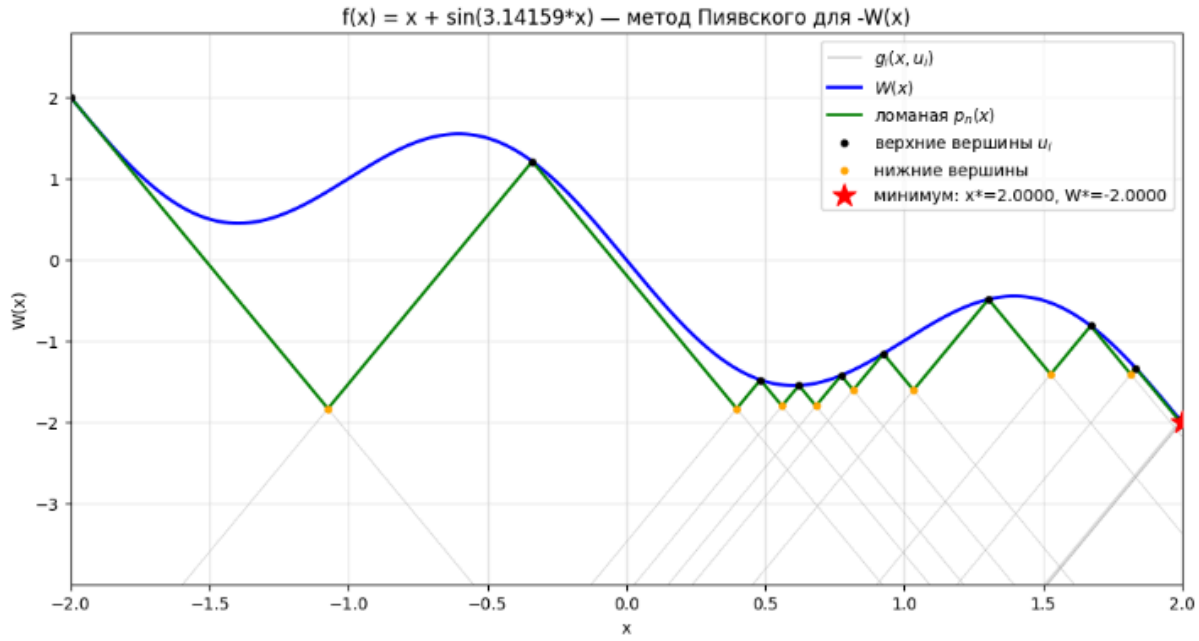
Для глобального максимума.

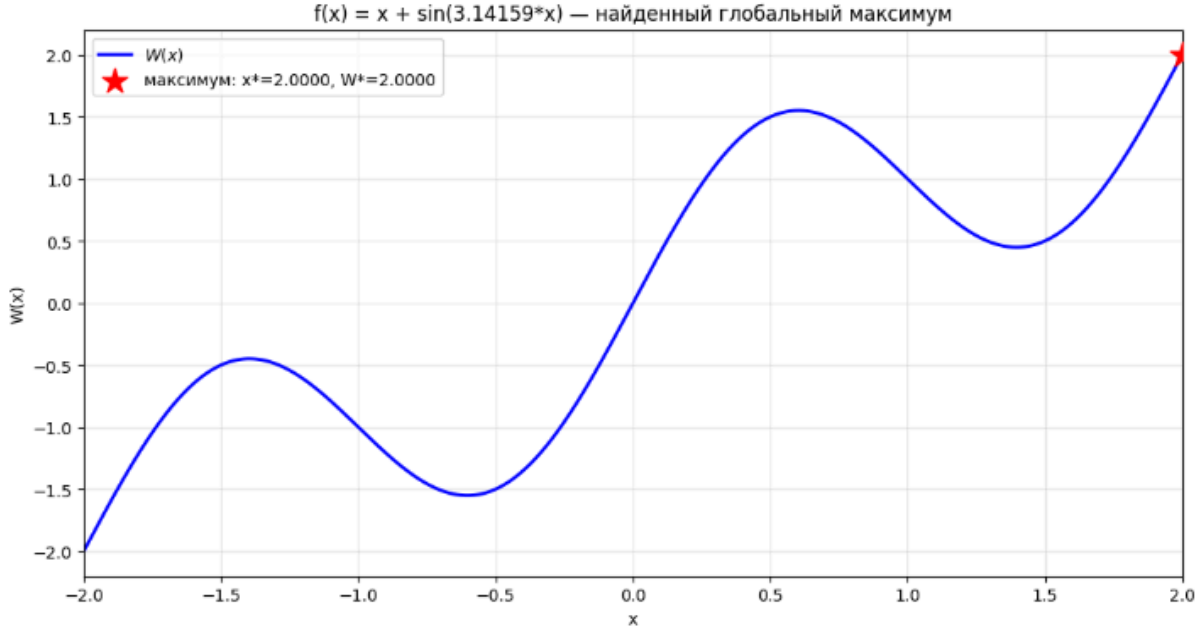
Так как при поиске глобального максимума мы умножаем функцию на -1, то будет некорректно отображать метод ломаных на исходной функции, поэтому было сделано 2 вывода: вывод исходной функции и вывод -W(x) с отображением метода Пиявского.

Функция из методички

```
 x + sin(3.14159*x)

ГЛОБАЛЬНЫЙ МАКСИМУМ
Функция: f(x) = x + sin(3.14159*x)
Отрезок: [-2, 2]
eps = 0.01
L = 4.14159
x* = 2.000000
W(x*) = 1.999995
Число итераций = 11
Статус = converged
Разрыв W(u_new)-p_new = 1.11791e-07
Время расчёта = 0.66 мс
```

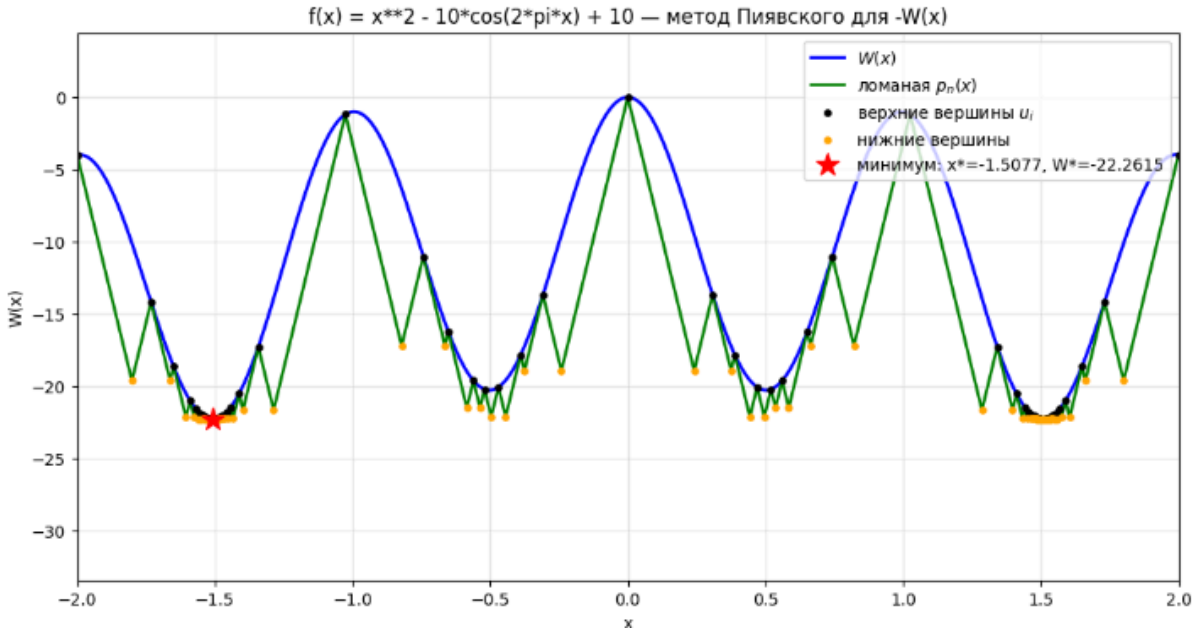


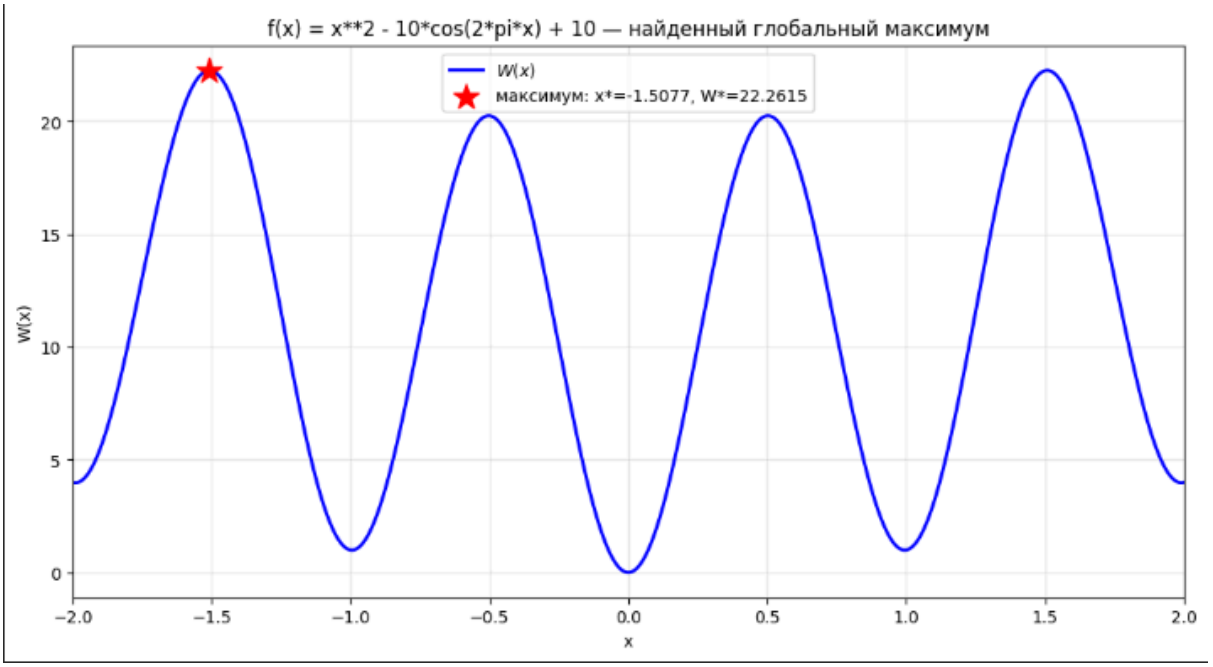


Функция Растригина

```
 x**2 - 10*cos(2*pi*x) + 10
 x**2 - 10*cos(2*pi*x) + 10
Автоматическая оценка L (reserve=1.2): 78.39917320442932

ГЛОБАЛЬНЫЙ МАКСИМУМ
Функция: f(x) = x**2 - 10*cos(2*pi*x) + 10
Отрезок: [-2, 2]
eps = 0.01
L = 78.39917320442932
x* = -1.507696
W(x*) = 22.261458
Число итераций = 296
Статус = converged
Разрыв W(u_new)-p_new = 0.00677908
Время расчёта = 89.86 мс
```

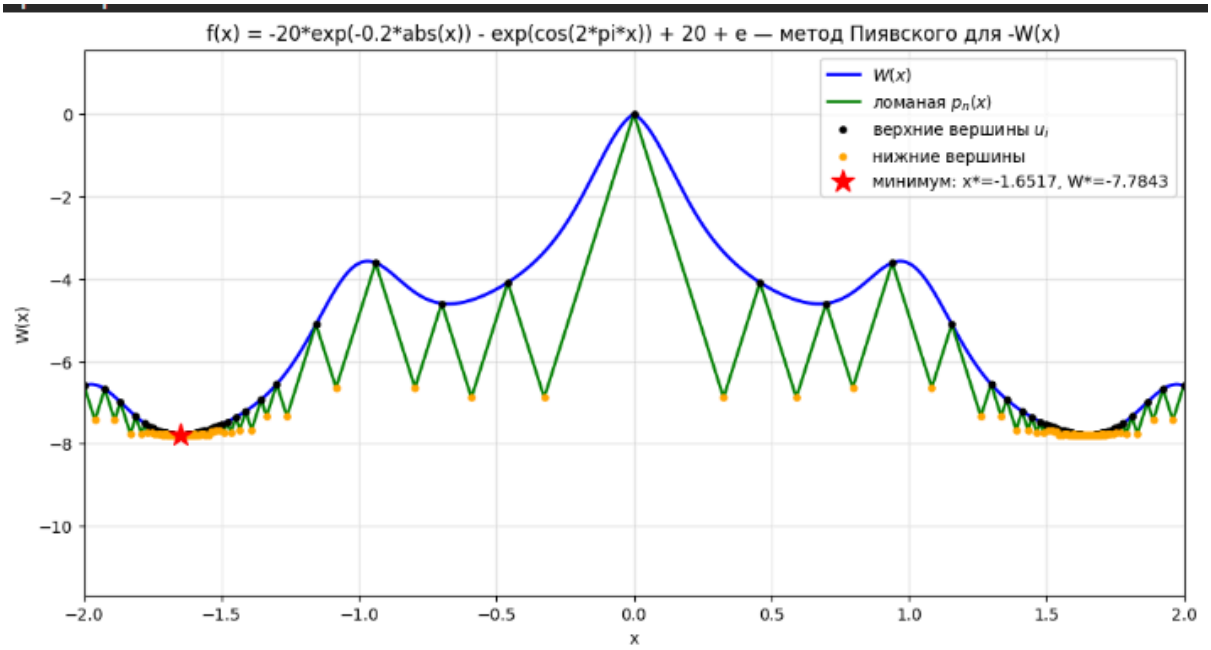


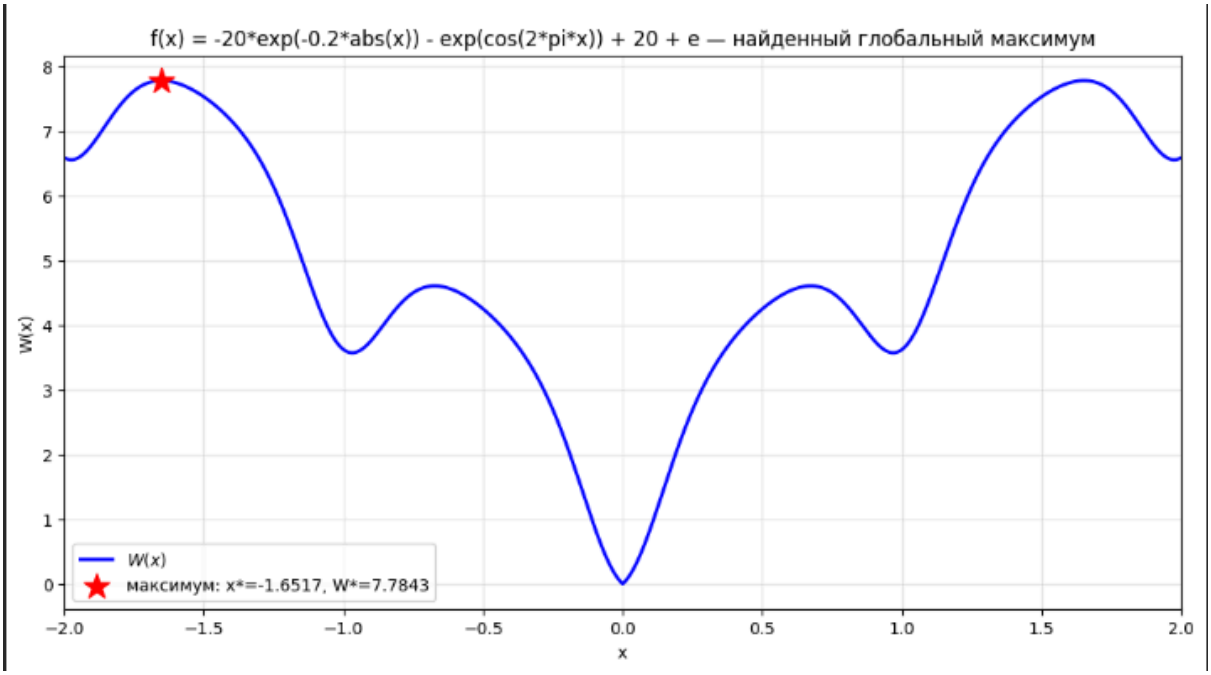


Функция Экли

```
 -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e

ГЛОБАЛЬНЫЙ МАКСИМУМ
Функция: f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e
Отрезок: [-2, 2]
eps = 0.01
L = 21.1
x* = -1.651681
W(x*) = 7.784298
Число итераций = 304
Статус = converged
Разрыв W(u_new)-p_new = 0.00602014
Время расчёта = 42.40 мс
```


f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e — метод Пиявского для -W(x)
W(x)
ломаная $p_n(x)$
верхние вершины u_i
нижние вершины
минимум: x*=-1.6517, W*=-7.7843
W(x)
x


f(x) = -20*exp(-0.2*abs(x)) - exp(cos(2*pi*x)) + 20 + e — найденный глобальный максимум
W(x)
максимум: x*=-1.6517, W*=7.7843
W(x)
x

Вывод по лабораторной работе

В ходе лабораторной работы был реализован метод Пиявского(ломаных) для поиска глобального экстремума одномерной функции на заданном отрезке.

В ходе тестирования программа успешно выполняла поиск глобальных минимумов и максимумов исследуемых функций.

Количество итераций зависит от вида функции и направления оптимизации. Для некоторых функций поиск максимума потребовал больше итераций, чем поиск минимума. Это связано с формой исследуемой функции и особенностями построения миноранты для преобразованной функции -W(x).

Во время работы метода новые точки выбираются не равномерно по всему отрезку, а в наиболее перспективных местах - в нижних вершинах текущей ломаной. Поэтому алгоритм последовательно уточняет область расположения глобального экстремума.

Ссылка на google_colab: ЛР2_метОпт_Степанова